

Pipeline:

Pobranie danych z API -> zapisanie danej do bucketa -> dodanie timestamp czasu przetworzenia -> zapisanie przetworzonych danych do bucketa -> usuwanie duplikatów -> obliczenie opóźnienia między pobieraniem danych a ich przetworzeniem -> wyświetlanie wykresów oraz statystyk

Różnica między near real time, a batch to:

Near Real Time - Aplikacja działa jako strumień przetwarzając nowe dane na bieżąco.

Batch - Aplikacje włącza się raz i analizuje ona wszystkie próbki.

Przykładowe statystyki:

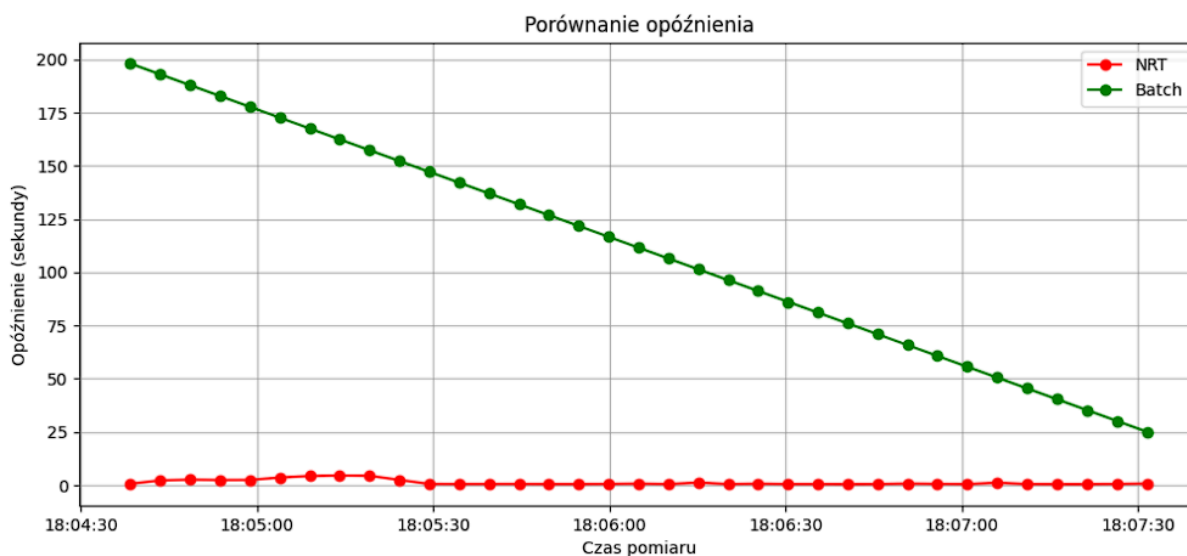
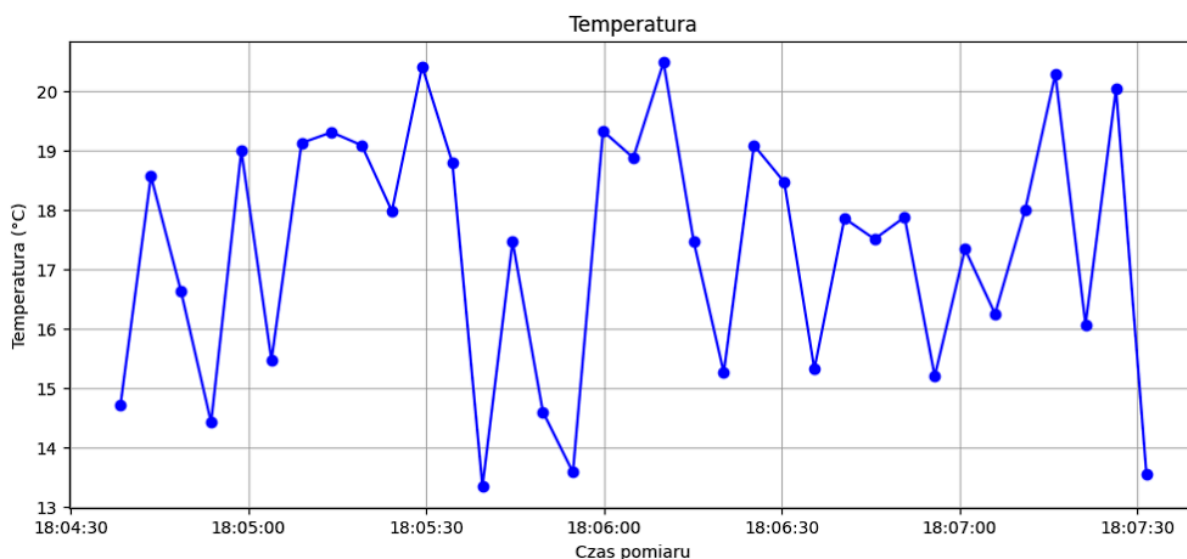
Unikalne rekordy: 35

Średnia temperatura: 17.34 °C

Średnie opóźnienie (NRT): 1.23 sekund

Średnie opóźnienie (Batch): 111.55 sekund

Przykładowe wyniki oraz porównanie opóźnień:



Złożoność architektoniczna:

Batch: Wczytuje surowe dane, wykonuje transformację i zapisuje je jednorazowo. Kod nie musi obsługiwać stanu.

Near Real-Time: Wykorzystuje silnik strumieniowy PySpark.

Złożoność zasobowa:

Batch: Może działać na klastrach tymczasowych, które uruchamiają się, wykonują zadanie i automatycznie wyłączają, co drastycznie obniża koszty. Jednorazowo większe zużycie zasobów niż NRT.

Near Real-Time: Wymaga klastra działającego w trybie ciągłym co powoduje mniejsze lecz stałe zużycie zasobów klastra w porównaniu do potoku Batch.

Złożoność utrzymaniowa:

Batch: Jeśli proces napotka awarie, po jej usunięciu uruchamia się od nowa dla całego dnia i nadpisuje dane.

Near Real-Time: Wymaga ciągłego monitoringu. Jeśli strumień napotka awarie, ponowne uruchomienie wymaga odczytania stanu z checkpointu.

Złożoność obliczeniowa:

Batch: Złożoność obliczeniowa rośnie wraz z rozmiarem historycznej bazy danych.

Near Real-Time: Złożoność obliczeniowa jest stała w czasie. PySpark przetwarza dane pojedynczo, więc każda operacja trwa podobną ilość czasu.

	Zalety	Wady
Near Real-Time	- Mniejsze zużycie zasobów - Niskie opóźnienia - Lepsza skalowalność	- Ciągłe zużycie zasobów
Batch	- Okresowe zużycie zasobów	- Większe zużycie zasobów - Wysokie opóźnienie - Gorsza skalowalność

Rekomendacja:

- Zaimplementowanie systemu automatycznego uruchamiania Batcha okresowo.

Instrukcja uruchomienia:

W celu poprawnego uruchomienia potoku NRT trzeba uruchomić wszystkie komórki w notebooku strumienia przetwarzania następnie zbieranie danych. Batch działa asynchronicznie i uruchamia się na żądanie.

Zaimplementowano regułę ekspercką danych wynikowych:

```
def classify_direction(direction):  
    if direction is None: return "Brak danych"  
    elif direction > 337.5: return "N"  
    elif direction > 292.5: return "NE"  
    elif direction > 247.5: return "E"  
    elif direction > 202.5: return "SE"  
    elif direction > 157.5: return "S"  
    elif direction > 112.5: return "SW"  
    elif direction > 67.5: return "W"  
    elif direction > 22.5: return "NW"  
    else: return "N"
```

Konkluzja:

Projekt wykazał, że architektura NRT jest kluczowa do natychmiastowego reagowania na dynamiczne zjawiska pogodowe ze względu na sekundowe opóźnienia przetwarzania, podczas gdy znacznie prostsza i tańsza struktura typu Batch pozostaje optymalnym rozwiązaniem do regularnych analiz zbiorczych danych historycznych.